

claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-ab50505f5b650a3bd.jsonl`
- SHA-256: `58ce0348f69e64d0dbe3bd3b8a167586c66ae61948b7ab06041fa3e894540021`
- Source modified: `2026-06-21T19:14:39+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`
```

Transcript

1. user (2026-06-21T19:12:53.043Z)

You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the REAL code the owner will deploy.

CRITICAL repo rules:

```
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared checkout owned by another session.
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master
-- 'PATHGLOB'          (search master's tree)
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH
(read a master file)
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i
--grep="..."         (history)
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you MAY use Read/Grep/Glob directly there.
```

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely. If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note. Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat; 'refuted' if wrong; 'unknown' if you genuinely could not determine it.

CLAIM (\$2.3): `python -m backend.main --server` runs the engine and binds 127.0.0.1:8000 (the safe default), "do NOT change the bind". Also packaging PR #296 (P0c) added a config/profile-driven bind address + DNS-rebinding Host-header guard.

On origin/master verify: (a) `python -m backend.main --server` is a valid invocation (find the --server arg parsing); (b) the DEFAULT bind host is 127.0.0.1 and default port 8000; (c) how #296's profile/config-driven bind works and whether anything could flip the default to 0.0.0.0 (a real exposure risk) ? and whether the Host-header guard is on by default. Report the exact config/env that controls the bind. This is a SAFETY claim ? be thorough.

2. assistant (2026-06-21T19:12:55.439Z)

I'll verify this safety claim by examining the engine's origin/master code. Let me start by reading the doc claim and then the relevant engine code.

3. user (2026-06-21T19:12:56.393Z)

```
1      # Deploy KhelSutra Studio for alpha ? a shareable page, no SSH
2
```

```

3      > **Status:** `READY` (owner executes the cloud steps) . **Topology:** split ? SPA on
Cloudflare Pages + engine on an `api.` cloudflared tunnel, Cloudflare Access over both (the
engine's `HOSTING_PLAN.md` $2 alpha plan). . **Cost:** ~$0/mo standing (Pages + Tunnel + Access
are free ?50 users) + Gemini per-run (~$0.05/match).
4
5      This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a
**page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email
allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
6
7      ## 0. The shape
8
9      ```
10     tester browser
11     ? https
12     ?
13     Cloudflare edge (free): DNS . TLS . Access (email-OTP allowlist) in front of BOTH
hostnames
14     ??? app.khelsutra.guru ? Cloudflare Pages          (the web/ SPA; built with
VITE_API_BASE)
15     ??? api.khelsutra.guru ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on
YOUR box
16                                     ? FastAPI engine
17                                     ? (Gemini
segmenter; GPU optional)
18                                     ? async jobs,
SQLite, output/ on disk
19     ```
20
21     - **No inbound ports.** `cloudflared` dials out; the engine stays bound to
`127.0.0.1:8000`. The box's home/public IP is never exposed.
22     - **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also*
verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a
direct hit to the tunnel can't spoof identity.
23     - **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ?
the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine
locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's
already env-driven).
24
25     > **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA
*and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).
26
27     ---
28
29     ## 1. Prerequisites
30
31     - A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU;
fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed;
the `badminton-highlight-indexer` engine checked out.
32     - A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing
site there).
33     - `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-
one/connections/connect-networks/downloads/`).
34     - A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks
to Google; this is a paid + cloud action).
35
36     ---
37
38     ## 2. Engine (the `api.` side)
39
40     ### 2.1 Install (with the auth extra)
41     ```bash
42     cd badminton-highlight-indexer
43     pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else

```

```

every request 500s
44     ``
45
46     ### 2.2 The safe-exposed config
47     Create / edit `config.json` on the box:
48     ``jsonc
49     {
50         "security": {
51             "edge_auth_enabled": true,                // turn the CF-Access JWT verify ON
52             "allowed_video_root": "/srv/khelsutra/incoming", // the path-jail ? REQUIRED when
exposed
53             "upload_dir": "/srv/khelsutra/incoming/uploads", // MUST sit UNDER
allowed_video_root (see §7)
54             "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
55             "cf_access_aud": "<the CF Access application AUD>" // filled in §4
56         }
57     }
58     ``
59     And export the env (a `.env` or your shell):
60     ``bash
61     export GEMINI_API_KEY="<your-rotated-key>"
62     export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru" # lock CORS to the SPA
origin
63     ``
64
65     > **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails
fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or
`cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a
half-configured exposure can't silently come up ? fix what it prints and restart.
66
67     ### 2.3 Run
68     ``bash
69     python -m backend.main --server          # binds 127.0.0.1:8000 (the safe default ? do NOT
change the bind)
70     ``
71     Keep it alive with a process manager (`systemd` on the droplet, mirroring
`site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).
72
73     ---
74
75     ## 3. The SPA on Cloudflare Pages (the `app.` side)
76
77     The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-
deploy on merge to `master`).
78
79     1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick
the `khelsutra` repo.
80     2. **Build settings:**
81         - **Root directory:** `web`
82         - **Build command:** `npm run build`
83         - **Build output directory:** `web/dist`
84         - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this
is what points the SPA at the engine tunnel (PR #64).
85     3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.
86
87     > Every push to `master` now rebuilds + redeploys the SPA. To preview a change first,
Pages builds a per-PR preview URL.
88
89     ---
90
91     ## 4. Cloudflare Access (gate BOTH hostnames)
92
93     1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
94     2. **Application domain:** add **both** `app.khelsutra.guru` **and**
`api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester

```

signs in once and both the SPA and its API calls are authorized).

```

95     3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50
users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
96     4. **Copy the application `AUD` tag** ? paste into the engine's `config.json`
`security.cf_access_aud` ($2.2), and set `cf_team_domain` to `https://<your-
team>.cloudflareaccess.com`. Restart the engine.
97     5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to
`api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass
options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is
redirected to the login page and every API call fails. The engine's CORS middleware
(`allow_credentials=false`, no cookies) then answers the preflight.
98
99     ---
100
101     ## 5. The tunnel (engine ? Cloudflare)
102
103     ```bash
104     cloudflared tunnel login                # browser auth to your zone
105     cloudflared tunnel create khelsutra-studio    # note the <TUNNEL_UUID>
106     # copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill
<TUNNEL_UUID>
107     cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
108     cloudflared tunnel run khelsutra-studio      # or install as a service:
`cloudflared service install`
109     ```
110     See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru` ?
http://127.0.0.1:8000).
111
112     ---
113
114     ## 6. Verify (the posture test + the CUJ)
115
116     **Exposure posture (must hold ? this is what makes it safe to share):**
117     - From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ?
**refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not
directly reachable.
118     - `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to
the Access login** (not a 200). ? The gate is in front.
119     - Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the
SPA loads and `/api/health` returns 200 (same session cookie). ?
120
121     **The shareable CUJ (what a tester does):** open `app.khelsutra.guru` ? sign in ? paste
a `gs://` video (or a path under `allowed_video_root`) ? **Process** ? watch progress ? pick
rallies ? **Build** ? **download the reel**. All from the page.
122
123     ---
124
125     ## 7. Honest caveats / deferred
126
127     - **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an
upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole).
Keep `upload_dir` inside the jail ($2.2).
128     - **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app
JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
129     - **Cross-origin reel download:** the reel streams from `api.` with `Content-
Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your
browser during the CUJ (cross-origin `` filename hints are ignored; the header is
the authority).
130     - **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large
matches upload as ?90 MB chunks (the SPA chunked-upload path), or pre-stage to a bucket and
ingest the `gs://` URI (already supported).
131     - **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine
offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this
with a queue (`HOSTING_PLAN.md` $3).
132     - **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for

```

```

clips you're OK sending.
133
134    ---
135
136    ## 8. Teardown
137    `cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages
custom domain; disable the Access application; stop the engine service. The box's `output/` +
`uploads/` retain originals ? sweep them per your retention policy before any wider exposure.
138
139    ---
140    ### Changelog
141    - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF
Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA
`VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative)
+ the engine `HOSTING_PLAN.md` §2.
142

```

4. user (2026-06-21T19:12:57.810Z)

```

origin/master:backend/main.py:866:    parser.add_argument("--server", action="store_true",
help="Run the FastAPI local API server")
origin/master:backend/main.py:981:    # CLI processing mode (no web UI needed). --app/--server
always run the server, never CLI single-shot.
origin/master:backend/main.py:1065:    # P2: the friendly launcher ? always localhost-
only (use --server for network exposure),

```

5. assistant (2026-06-21T19:13:00.430Z)

Let me read the relevant sections of main.py on origin/master to understand the `--server` invocation and bind defaults.

6. user (2026-06-21T19:13:02.024Z)

```

# CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
if args.video_path and not args.server and not args.app:
    print(f"[CLI] Processing video file: {args.video_path}")
    # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
    # The local-existence fast-fail only applies to a local input (stat the RESOLVED key,
not the
    # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error, not a
crash.
    storage_cfg = getattr(global_config, "storage", None)
    try:
        input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
    except ValueError as e:
        print(f"[FAIL] {e}")
        return
    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
        print(f"[FAIL] Video file not found: {args.video_path}")
        return
    local_video = storage.acquire_local_input(args.video_path, storage_cfg)

    video_id = compute_video_id(local_video)
    was_reingest = db_instance.get_video(video_id) is not None

    # Validate (with-context variant surfaces ffprobe_meta for the report)
    print("[CLI] Validating...")
    val_results, val_context = registry.run_all_with_context(local_video, global_config)
    failures = [r for r in val_results if not r.passed]

    # Save video status
    status = "failed" if failures else "processed"

```

```

        db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])

    seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
    segmenter_actual = None
    segmentation_ran = False
    try:
        print("\n" + "="*40)
        print("VALIDATION SUMMARY")
        print("="*40)
        for r in val_results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"{status_symbol} {r.validator_name}: {r.message}")
        print("="*40 + "\n")

        if failures:
            print("[FAIL] Video failed checks. Indexing halted.")
            return

        # Index
        segmenter_cls = segmenter_registry.get(seg_name)
        if not segmenter_cls:
            print(f"[FAIL] Segmenter '{seg_name}' not found.")
            return

        print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
        segmenter_actual = seg_name
        play_wm, cand_wm = db_instance.segment_watermarks(video_id)
        segmenter = segmenter_cls(db_instance, global_config)
        result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
        segmentation_ran = True

        if isinstance(result, Failure):
            print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
            print("[FAIL] Indexing halted for video.")
            segments = []
        else:
            segments = result.value if hasattr(result, "value") else result
            print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
database: {db_path}")
            # Destroy-AFTER-confirm: keep new on success, restore prior on empty/failure.
            _finalize_reingest(video_id, segments, play_wm, cand_wm)
        finally:
            # Always emit the per-video observability report. Never raises.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths:
                print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                print(f"[REPORT] Machine JSON      : {paths.get('json')}")
            return

    # Start Local Server Mode
    if args.app or args.server or not args.video_path:
        if args.app:
            # P2: the friendly launcher ? always localhost-only (use --server for network
exposure),
            # auto-pick a free port if --port is busy, and open the bundled UI once the server
is up.

```

```

bind_host = "127.0.0.1"
port = _resolve_app_port(bind_host, args.port)
if port != args.port:
    print(f"[APP] Port {args.port} is busy ? using {port} instead.")
url = f"http://{bind_host}:{port}/"
# The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback staying
in
# the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-loopback-
only set
# with --app, or the poll 400s and the browser just won't auto-open ? the server
still runs).
print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
      f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
threading.Thread(target=_open_browser_when_ready, args=(url, url + "api/health"),
                  daemon=True).start()
else:
    bind_host = _resolve_bind_host(global_config, args.host) # P0c: config/profile-
driven (default 127.0.0.1)
    port = args.port
    print(f"[SERVER] Starting local FastAPI server on http://{bind_host}:{port}...")
    if bind_host not in ("127.0.0.1", "localhost", "::1"):
        print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ? ensure
an auth "
              f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
              f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
              f"loopback-only and will 400 forwarded requests otherwise).")
    # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3): the
    # in-process wake-timer for any parked 'waiting' job died with the old process, so
    # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
    _get_executor().submit(_drain_due_jobs)
    uvicorn.run(app, host=bind_host, port=port)

if __name__ == "__main__":
    # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads` resolve
the
    # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ? a
SEPARATE
    # module object. If `main()` runs here it configures `__main__`'s globals, leaving
    # `backend.main.db_instance` = None; the async-job drain then crashes silently on
    # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch through
    # `backend.main` so `main()` configures the module everyone actually reads. (No recursion:
    # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-fire.)
    import backend.main as _bm

    _bm.main()

```

7. user (2026-06-21T19:13:03.242Z)

```

origin/master:backend/main.py:13:import uvicorn
origin/master:backend/main.py:33:from starlette.middleware.trustedhost import
TrustedHostMiddleware
origin/master:backend/main.py:68:     does zero filesystem I/O ? a hosted/multi-tenant deploy
sets this to lock CORS to its origin."""
origin/master:backend/main.py:77:# off. The origin list is configurable via the env var above
(default ["*"]) so a hosted deploy can
origin/master:backend/main.py:88:def _allowed_hosts_from_env(env: Optional[Dict[str, str]] =
None) -> List[str]:
origin/master:backend/main.py:89:     """P0c: Host-header allowlist ? a DNS-rebinding guard for
the localhost appliance. A malicious
origin/master:backend/main.py:90:     web page can point a domain it controls at 127.0.0.1 and
script requests to the local server;
origin/master:backend/main.py:93:     Starlette's TestClient host ``testserver`` so the in-
process suite isn't broken. Read from the
origin/master:backend/main.py:95:     deploy sets its public host (or ``*`` when fronted by an

```

```

auth gate that validates the host)."""
origin/master:backend/main.py:98:     hosts = [h.strip() for h in raw.split(",") if h.strip()]
origin/master:backend/main.py:99:     # IPv4 loopback (the default bind) + the TestClient host.
NB: Starlette's TrustedHostMiddleware
origin/master:backend/main.py:100:    # matches on host.split(":")[0], which mangles a bracketed
IPv6 Host ("[::1]:port"), so IPv6
origin/master:backend/main.py:101:    # loopback is NOT supported by this guard ? access the
appliance via 127.0.0.1/localhost.
origin/master:backend/main.py:102:    return hosts or ["localhost", "127.0.0.1", "testserver"]
origin/master:backend/main.py:105:# DNS-rebinding guard. Bound to localhost by default, so only
loopback Host headers are honoured
origin/master:backend/main.py:107:# off so an unexpected ``www`` host is rejected rather than
307-redirected.
origin/master:backend/main.py:110:    allowed_hosts=_allowed_hosts_from_env(),
origin/master:backend/main.py:115:def _resolve_bind_host(config: Optional[AppConfig] = None,
cli_host: Optional[str] = None) -> str:
origin/master:backend/main.py:116:    """P0c: the server bind address. Precedence: ``--host`` >
``RALLY_BIND_HOST`` env > ``0.0.0.0``
origin/master:backend/main.py:118:    in front) > ``127.0.0.1`` (default: localhost-only,
today's behaviour). Binding a routable
origin/master:backend/main.py:119:    interface with no auth gate is unsafe, hence the localhost
default; widen it consciously."""
origin/master:backend/main.py:120:    if cli_host and cli_host.strip():
origin/master:backend/main.py:121:        return cli_host.strip()
origin/master:backend/main.py:127:        return "0.0.0.0" # behind a Cloudflare-Access-style
edge gate (M9) ? bind all interfaces
origin/master:backend/main.py:128:    return "127.0.0.1"
origin/master:backend/main.py:133:def _port_is_free(host: str, port: int) -> bool:
origin/master:backend/main.py:134:    """True if (host, port) can be bound right now (best-
effort; a TOCTOU race is acceptable ?
origin/master:backend/main.py:135:    uvicorn surfaces a real bind failure loudly)."""
origin/master:backend/main.py:138:        s.bind((host, port))
origin/master:backend/main.py:144:def _find_free_port(host: str) -> int:
origin/master:backend/main.py:145:    """An OS-assigned free port on host."""
origin/master:backend/main.py:147:        s.bind((host, 0))
origin/master:backend/main.py:151:def _resolve_app_port(host: str, port: int) -> int:
origin/master:backend/main.py:153:    return port if _port_is_free(host, port) else
_find_free_port(host)
origin/master:backend/main.py:587:    # input: its null-byte guard always applies, and a
configured allowed_video_root (hosted mode)
origin/master:backend/main.py:868:                                help="Launch the app: start the local
server (localhost-only) and open the "
origin/master:backend/main.py:871:    parser.add_argument("--host", default=None,
origin/master:backend/main.py:872:                        help="Bind address. Default 127.0.0.1
(localhost-only); 0.0.0.0 when "
origin/master:backend/main.py:1065:    # P2: the friendly launcher ? always localhost-
only (use --server for network exposure),
origin/master:backend/main.py:1067:    bind_host = "127.0.0.1"
origin/master:backend/main.py:1068:    port = _resolve_app_port(bind_host, args.port)
origin/master:backend/main.py:1071:    url = f"http://{bind_host}:{port}/"
origin/master:backend/main.py:1072:    # The browser-open health poll sends Host:
127.0.0.1 ? it relies on loopback staying in
origin/master:backend/main.py:1080:    bind_host = _resolve_bind_host(global_config,
args.host) # P0c: config/profile-driven (default 127.0.0.1)
origin/master:backend/main.py:1082:    print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
origin/master:backend/main.py:1083:    if bind_host not in ("127.0.0.1", "localhost",
"::1"):
origin/master:backend/main.py:1084:        print(f"[SERVER] ? Binding {bind_host}
exposes the API on the network ? ensure an auth "
origin/master:backend/main.py:1086:                                f"RALLY_ALLOWED_HOSTS to your public
host(s) (the Host-header guard defaults to "
origin/master:backend/main.py:1092:    uvicorn.run(app, host=bind_host, port=port)

```


This is the core of the safety claim. Let me read the `\_resolve\_bind\_host` function in full and the arg parsing, plus the port default.

9. user (2026-06-21T19:13:07.566Z)

```
from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)
```

```
app = FastAPI(title="Sports Highlight Indexer API")
```

```
def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the `RALLY_CORS_ALLOW_ORIGINS` env var (comma-separated),
    default `["*"]`. Read from the ENV (not a cwd `config.json`) so importing
    ``backend.main``
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
    origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
    origins = [o.strip() for o in raw.split(",") if o.strip()]
    return origins or ["*"]
```

CORS for the web UI. `allow_origins='*' with allow_credentials=True is rejected by browser fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials off. The origin list is configurable via the env var above (default ["*"]) so a hosted deployment lock it to its origin; the middleware is fixed at startup.`

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins_from_env(),
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
    malicious
    web page can point a domain it controls at 127.0.0.1 and script requests to the local
    server;
    the browser sends `Host: attacker.com`, which this rejects (the server only answers
    loopback
    Host headers by default). `RALLY_ALLOWED_HOSTS` (comma-separated) overrides; the default
    keeps
    Starlette's TestClient host `testserver` so the in-process suite isn't broken. Read from
    the
    ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
    routable/edge
    deploy sets its public host (or ``*`` when fronted by an auth gate that validates the
    host)."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
    hosts = [h.strip() for h in raw.split(",") if h.strip()]
    # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
    TrustedHostMiddleware
    # matches on host.split(":")[0], which mangles a bracketed IPv6 Host (":::1:port"), so IPv6
    # loopback is NOT supported by this guard ? access the appliance via 127.0.0.1/localhost.
    return hosts or ["localhost", "127.0.0.1", "testserver"]
```

DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are

(a remote page can't read responses cross-origin AND its Host header is rejected here off so an unexpected ``www.`` host is rejected rather than 307-redirected.

```
app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=_allowed_hosts_from_env(),
    www_redirect=False,
)
```

```
def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] = None) -> str:
    """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
    ``0.0.0.0``
    when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS an auth
    gate
    in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a routable
    interface with no auth gate is unsafe, hence the localhost default; widen it consciously."""
    if cli_host and cli_host.strip():
        return cli_host.strip()
    env = os.environ.get("RALLY_BIND_HOST")
    if env and env.strip():
        return env.strip()
    sec = getattr(config, "security", None)
    if getattr(sec, "edge_auth_enabled", False):
        return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
    interfaces
    return "127.0.0.1"
```

--- P2: the friendly `indexer --app` launcher (cross-platform, no native component ? O1

```
def _port_is_free(host: str, port: int) -> bool:
    """True if (host, port) can be bound right now (best-effort; a TOCTOU race is acceptable ?
    uvicorn surfaces a real bind failure loudly)."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        try:
            s.bind((host, port))
            return True
        except OSError:
            return False

def _find_free_port(host: str) -> int:
    """An OS-assigned free port on host."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((host, 0))
        return int(s.getsockname()[1])

def _resolve_app_port(host: str, port: int) -> int:
    """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never blocks
    the app."""
    return port if _port_is_free(host, port) else _find_free_port(host)

def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
    """Poll `health_url` until the server is up, then open `url` in the user's default browser.
    Returns True if a browser open was attempted. No-op (returns False) when RALLY_NO_BROWSER is
    set
    (tests / headless) or the server never becomes healthy (don't open a dead page). Never
    raises."""
    if os.environ.get("RALLY_NO_BROWSER"):
```

10. user (2026-06-21T19:13:08.156Z)

```
def main():
    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
    parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
setup utility")
    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip
to run detailed diagnostics on")
    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
    parser.add_argument("--app", action="store_true",
                        help="Launch the app: start the local server (localhost-only) and open
the "
                        "bundled UI in your browser. Falls back to a free port if --port is
busy.")
    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server
on")
    parser.add_argument("--host", default=None,
                        help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
                        "security.edge_auth_enabled. Expose on a network ONLY behind an
auth gate.")
    parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")

    available_segmenters = list(segmenter_registry.list_available().keys())
    parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
    parser.add_argument("--trim-start", type=float, help="Start time in seconds for the debug
trim segment feature")
    parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug trim
segment feature")
    parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")

    args = parser.parse_args()

    if args.setup:
        # Invoke the diagnostics/bootstrap script (renamed from the old root
        # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
        # build system). Resolve its path from this file so `indexer --setup`
        # works regardless of the caller's cwd.
        import subprocess
        doctor_path = os.path.join(
            os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
            "scripts", "doctor.py",
        )
        print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
        subprocess.run([sys.executable, doctor_path])
        return

    if args.trim_start is not None and args.trim_end is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to extract a segment.")
```

11. assistant (2026-06-21T19:13:12.272Z)

Now I have the complete picture. This is a critical finding for the safety claim. Let me verify the key detail: the doc says `--server` "binds 127.0.0.1:8000 (the safe default ? do NOT change the bind)", but the \$2.2 config the doc instructs the owner to create sets `edge\_auth\_enabled: true`. And `\_resolve\_bind\_host` returns `0.0.0.0` when `edge\_auth\_enabled` is true. Let me verify the config loading to confirm `global\_config.security.edge\_auth\_enabled` reflects the config.json.

12. user (2026-06-21T19:13:14.472Z)

```
origin/master:backend/config/models.py:280:class SecurityConfig(BaseModel):
origin/master:backend/config/models.py:297:     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT
is required/verified (the local single-user
origin/master:backend/config/models.py:303:     edge_auth_enabled: bool = False
origin/master:backend/config/models.py:404:     security: SecurityConfig =
Field(default_factory=SecurityConfig)
origin/master:backend/config/startup.py:19:behind an edge gate (the owner flips
``security.edge_auth_enabled`` on for the Cloudflare-Access
origin/master:backend/config/startup.py:30:a strict **no-op** ? exactly the pre-M5 behaviour;
with ``edge_auth_enabled=False`` (today's default)
origin/master:backend/config/startup.py:91:     """Exposure-safety posture for an *exposed* HTTP
server. Gated on ``security.edge_auth_enabled``
origin/master:backend/config/startup.py:97:     sec = getattr(config, "security", None)
origin/master:backend/config/startup.py:98:     if not sec or not getattr(sec,
"edge_auth_enabled", False):
origin/master:backend/config/startup.py:110:         "security.edge_auth_enabled is ON
(engine exposed) but security.allowed_video_root is "
origin/master:backend/config/startup.py:124:         f"security.edge_auth_enabled is ON but
{missing} is not configured ? the Cf-Access JWT "
origin/master:backend/config/startup.py:125:         "cannot be verified (every request would
fail). Set security.cf_team_domain "
origin/master:backend/config/startup.py:126:         "(e.g.
https://<team>.cloudflareaccess.com) and security.cf_access_aud (the CF Access "
origin/master:backend/config/startup.py:136:         "security.edge_auth_enabled is ON but
PyJWT[crypto] is not importable ? Cf-Access "
origin/master:backend/config/startup.py:146:         "security.edge_auth_enabled is ON but
RALLY_ALLOWED_HOSTS is unset ? the Host-header guard "
origin/master:backend/config/startup.py:167:     DEFAULT-OFF: no profile ? no profile checks;
``edge_auth_enabled=False`` ? no exposure checks.
origin/master:backend/config/startup.py:174:     # runs with no profile set), gated on
edge_auth_enabled (default-OFF). Server-only.
```

13. user (2026-06-21T19:13:15.461Z)

```
    print("[ERROR] Please provide --video-path to extract a segment.")
    return

# Determine output path
out_filename = args.trim_output or "trimmed_segment.mp4"
if os.path.isabs(out_filename):
    out_path = out_filename
    out_dir = os.path.dirname(out_path)
    out_name = os.path.basename(out_path)
else:
    out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-identical
when unset)
    out_path = os.path.join(out_dir, out_filename)
    out_name = out_filename

os.makedirs(out_dir, exist_ok=True)
print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s from
{args.video_path}...")

# global_config isn't initialized this early; load the typed config so the trim
# clip carries the same configured stitching paddings + watermark as a real
# highlight (watermark default-on).
stitching = load_app_config().stitching
compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                        end_padding=stitching.end_padding,
watermark=stitching.watermark)
try:
    res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
```

```

        print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
    except Exception as e:
        print(f"[ERROR] Failed to compile trimmed segment: {e}")
    return

if args.debug_clip is not None:
    if not args.video_path:
        print("[ERROR] Please provide --video-path to debug a specific clip.")
        return
    run_cli_diagnose_clip(args.video_path, args.debug_clip)
    return

# Initialize Global Config and DB
global global_config, db_instance
global_config = load_app_config() # returns typed AppConfig
_enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE profile

# The CLI --output-dir overrides the configured output directory. It now lives
# on the typed model (OutputConfig.output_dir), so every consumer ? including
# /api/compile ? reads the same value instead of a write-only runtime hack.
# P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
# unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to before.
global_config.output.output_dir = resolve_output_dir(args.output_dir)
os.makedirs(global_config.output.output_dir, exist_ok=True)

db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

# P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job sweep
below.
# The job worker is in-process, so a concurrent process opening this DB would sweep THIS
one's
# in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-released on
exit
# or crash) is keyed to db_path, so a different --output-dir/DB still runs concurrently.
global _instance_lock
try:
    _instance_lock = acquire_lock(db_path + ".lock")
except OSError as exc:
    # The lock path isn't writable (read-only dir / perms). Don't crash and don't falsely
claim a
    # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure must not
block a
    # legitimate single instance).
    logger.warning("[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
                    "WITHOUT the second-instance guard.", db_path, exc)
    _instance_lock = None
else:
    if _instance_lock is None:
        print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
              f"start ? a second instance would corrupt in-flight job state. Stop the other
one, or "
              f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
    sys.exit(1)

db_instance = Database(db_path)

# Crash recovery (#16): jobs left queued/running by a previous process are dead ?
# the executor is in-process. Mark them failed so pollers see a terminal state.
swept = db_instance.fail_stale_jobs()
if swept:
    logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

```

```

# CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
if args.video_path and not args.server and not args.app:
    print(f"[CLI] Processing video file: {args.video_path}")
# M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
# The local-existence fast-fail only applies to a local input (stat the RESOLVED key,
not the

```

14. assistant (2026-06-21T19:13:19.024Z)

Now let me confirm how config.json is loaded (whether `load_app_config()` reads the config.json the doc instructs the owner to create), and check the SecurityConfig fields.

15. user (2026-06-21T19:13:20.708Z)

```

class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]

    # --- M9 edge auth (Cloudflare Access, D9) ---
    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
    # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
    Assertion`
    # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
    identity
    # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
    # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
    Access
    # application's audience tag. NO secrets here ? these are public identifiers.
    edge_auth_enabled: bool = False
    cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs
    JWKS)
    cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
    # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at import in
    # backend/main.py so importing the app does no filesystem I/O; default ["*"]). Credentials
    stay
    # OFF ? Cf-Access is a header, not a cookie.)

class StorageConfig(BaseModel):
    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
    behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0 Spaces
    + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a locally
    MOUNTED
    Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to the
    constructor ?
    **secrets are never stored here**, only endpoints / regions / file paths / source-selectors
    (boto3/google auth chains supply credentials). See ``backend/storage/``.
    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
    # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
    output_root: str = ""

```

16. user (2026-06-21T19:13:21.885Z)

```
origin/master:backend/config/loader.py:4:merged with optional file overrides (config.json).
origin/master:backend/config/loader.py:33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and
equals ``Path("config.json")`` (CWD-relative) when
origin/master:backend/config/loader.py:34:# the env is unset, exactly the pre-P0a value. (The
old module-level ``DEFAULT_CONFIG_PATH`` constant
origin/master:backend/config/loader.py:41: replace). Used ONLY for the profile-preset
precedence path so a config.json sub-key
origin/master:backend/config/loader.py:54: ""The EXPLICITLY chosen deployment profile, env
winning over config.json. ``"" if none
origin/master:backend/config/loader.py:60: asserting a preset that was never applied. A bad
value in config.json is left to surface as
origin/master:backend/config/loader.py:69: "and falling back to config.json
deployment.profile.", ep, "", ".join(PROFILE_PRESETS),
origin/master:backend/config/loader.py:130: # A config.json that is valid JSON but not an
OBJECT (e.g. ``[]``, ``42``, ``"x"`` , ``null``) would
origin/master:backend/config/loader.py:149: # Precedence: built-in defaults < profile preset
< config.json (< env/--set, applied at
origin/master:backend/config/loader.py:151: # explicitly selected (DEPLOYMENT_PROFILE env or
config.json deployment.profile). With no
origin/master:backend/config/loader.py:157: # Only an unrecognised value from config.json
reaches here (env typos already fell back
origin/master:backend/config/loader.py:168: # Record the profile actually applied ? the
env may have chosen it over config.json,
origin/master:backend/config/loader.py:176: # An active profile with no explicit
compute_target (e.g. config.json blanked it
origin/master:backend/config/loader.py:183: "config.json overrides it to %r ?
honoring the explicit override.",
origin/master:backend/config/loader.py:203: ""Write a fresh default config.json.
origin/master:backend/config/models.py:6:The models support graceful migration from old
config.json files via extra="allow".
origin/master:backend/config/models.py:241: (under `stitching.watermark` in config.json) to
turn it off. The text is fully
origin/master:backend/config/models.py:353: behaves exactly as before this section existed. A
profile is opt-in (``config.json``
origin/master:backend/config/models.py:363: # Where the core runs. Set by the profile preset;
overridable in config.json. Inert in M1
origin/master:backend/config/models.py:410: # Allow extra keys during transition so existing
config.json files do not break.
```

17. assistant (2026-06-21T19:13:26.159Z)

I now have everything to verify all three sub-claims. The critical finding is clear. Let me confirm the doc text precisely ? the \$2.2 config sets `edge\_auth\_enabled: true`, and \$2.3 immediately after says `--server` binds 127.0.0.1. But `resolve\_bind\_host` returns `0.0.0.0` when `edge\_auth\_enabled` is true. Let me verify `load\_app\_config` returns the merged config (so config.json's `edge\_auth\_enabled: true` reaches `global\_config.security`).

18. user (2026-06-21T19:13:28.765Z)

```
---loader head---
file_data: dict[str, Any] = {}
if cfg_path.exists():
    try:
        with open(cfg_path, "r", encoding="utf-8") as f:
            file_data = json.load(f)
    except Exception as exc:
        # Non-fatal: continue with defaults + whatever we could parse.
        # In production we might log here.
        logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")

# A config.json that is valid JSON but not an OBJECT (e.g. ``[]``, ``42``, ``"x"`` , ``null``) would
# otherwise crash startup: a truthy non-mapping hits `.items()` in _unknown_key_paths, and
any
```

```

# non-mapping breaks the `**defaults, **file_data` unpack. Degrade to defaults + warn, per
the
# loader's "bad input is non-fatal" contract.
if not isinstance(file_data, dict):
    logger.warning("%s is not a JSON object (got %s); ignoring it and using defaults.",
        cfg_path, type(file_data).__name__)
    file_data = {}

if file_data:
    unknown = _unknown_key_paths(file_data, AppConfig)
    if unknown:
        logger.warning(
            "[CONFIG WARNING] %s contains keys the typed config does not "
            "recognize (typo'd or removed knobs ? top-level extras are kept "
            "but unread; unknown section keys are silently dropped): %s",
            cfg_path, ", ".join(sorted(unknown)),
        )

# Precedence: built-in defaults < profile preset < config.json (< env/--set, applied at
# consumption sites downstream). A *deployment profile* is opt-in ? it is applied ONLY when
# explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile). With no
# profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
defaults = _get_builtin_defaults()
profile = _resolve_explicit_profile(file_data)

if profile and not is_known_profile(profile):
    # Only an unrecognised value from config.json reaches here (env typos already fell back
    # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits the
    # typed Literal at model_validate below, a hard clear error ? loud, never silent.
    logger.warning(
        "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no preset.",
        profile, ", ".join(PROFILE_PRESETS),
    )
    profile = ""

if profile:
    merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
    # Record the profile actually applied ? the env may have chosen it over config.json,
    # so re-stamp it onto the merged deployment section to keep the record honest.
    merged.setdefault("deployment", {})
    if isinstance(merged["deployment"], dict):
        merged["deployment"]["profile"] = profile
        ct = merged["deployment"].get("compute_target")
        canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
        if not ct and canonical:
            # An active profile with no explicit compute_target (e.g. config.json blanked it
            # to "") gets its canonical target back ? the record never silently lies about
an
            # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
            merged["deployment"]["compute_target"] = canonical
        elif ct and canonical and ct != canonical:
            logger.warning(
                "[CONFIG] deployment profile %r normally uses compute_target=%r but "
                "config.json overrides it to %r ? honoring the explicit override.",
                profile, canonical, ct,
            )
            logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
                profile, merged["deployment"].get("compute_target"))
    else:
        # No profile ? the original shallow merge (Pydantic re-defaults nested sections).
        merged = {**defaults, **file_data}
        suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only (D15)
        if suggestion:

```



```

        logger.debug(
            "[CONFIG] no deployment.profile set (manual knobs). Environment suggests %r ? "
            "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect never "
            "auto-applies ? D15).", suggestion,
        )

    return AppConfig.model_validate(merged)

def save_default_config(path: str | Path | None = None) -> Path:
    """Write a fresh default config.json.

    Used by scripts/doctor.py diagnostics and the --setup CLI path.

```

19. assistant (2026-06-21T19:13:35.809Z)

Confirmed. `load_app_config()` merges `config.json` into the typed `AppConfig`, so the §2.2 `config.json` with `"security": {"edge_auth_enabled": true, ...}` will set `global_config.security.edge_auth_enabled = True`. Then at the `--server` branch (line 1080), `_resolve_bind_host(global_config, args.host)` is called with `args.host = None` (no `--host` given) and no `RALLY_BIND_HOST` env in the doc, so it falls through to the `edge_auth_enabled` check and returns `**0.0.0.0**`, NOT `127.0.0.1`.

I have a clear, concrete picture of all three sub-claims. Let me record my findings.

The verdict is `**partial**` (mostly the bind safety story is real and well-engineered, but the doc's specific claim that following §2.2 + §2.3 binds `127.0.0.1` is REFUTED by the code ? it binds `0.0.0.0`). This is a real exposure-relevant discrepancy worth flagging as a blocker.

20. user (2026-06-21T19:14:00.245Z)

```
<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be
parsed as JSON.
You sent (first 200 of 4445 bytes): {"claim": "§2.3: `python -m backend.main --server` runs the
engine and binds 127.0.0.1:8000 (the safe default ? \"do NOT change the bind\"); PR #296 (P0c)
added a config/profile-driven bind + DNS-rebi
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters,
or truncated output. Retry with valid JSON.</tool_use_error>
```

21. assistant (2026-06-21T19:14:02.914Z)

Let me call `StructuredOutput` with the proper structured fields rather than a raw JSON string.

22. user (2026-06-21T19:14:24.573Z)

Structured output provided successfully

23. assistant (2026-06-21T19:14:39.511Z)

I've completed the verification. Summary of findings for the §2.3 bind safety claim, verified against ENGINE origin/master:

****Sub-claim (a) ? ``python -m backend.main --server`` is valid: CONFIRMED.**** `backend/main.py:866`` defines the `--server` flag; the server branch at `main.py:1080-1092`` runs `uvicorn`.

****Sub-claim (b) ? default bind `127.0.0.1:8000`: PARTIAL.**** Port 8000 default is confirmed (`main.py:870``). But `127.0.0.1` is the default ONLY when `edge_auth_enabled`` is OFF and neither `--host`` nor `RALLY_BIND_HOST`` is set (`_resolve_bind_host``, `main.py:115-128``).

****Sub-claim (c) ? #296 config/profile-driven bind + Host-header guard: CONFIRMED, and it surfaces the real exposure subtlety.**** `_resolve_bind_host`` returns `**0.0.0.0`` when `security.edge_auth_enabled`` is `True``. The `TrustedHostMiddleware`` DNS-rebinding guard (`main.py:88-111``) is present and ON by default with a loopback-only allowlist, overridable via `RALLY_ALLOWED_HOSTS``.

****The blocker discrepancy:**** The runbook's §2.2 instructs the owner to create a `config.json` with `edge\_auth\_enabled: true`, and §2.3 immediately claims `--server` "binds 127.0.0.1:8000 (the safe default ? do NOT change the bind)." But with that exact config, the code binds ****0.0.0.0****, not 127.0.0.1 (`main.py:127` + `loader.py` merges config.json into `global\_config.security`). The deploy is still safe in practice (outbound-only cloudflared tunnel + in-app Cf-Access JWT verify + Host guard), but the doc's literal 127.0.0.1 / "no inbound port via loopback bind" framing is wrong.

Two owner actions the doc omits: (1) the §6 `curl public-ip:8000 ? refused` posture test relies on no inbound firewall port being open (NOT on the loopback bind, since the socket is 0.0.0.0); (2) the §2.2 config block does not set `RALLY\_ALLOWED\_HOSTS`, so tunneled requests arriving with `Host: api.khelsutra.guru` will be 400'd by the loopback-only Host guard until it's set. Setting `RALLY\_BIND\_HOST=127.0.0.1` would make the doc's literal claim true while still working through the tunnel.

The relevant files: `C:\Users\avidu\Projects\khelsutra\deploy\DEPLOY\_ALPHA.md` (§2.2/§2.3/§6) and ENGINE `backend/main.py` (`\_resolve\_bind\_host`, `\_allowed\_hosts\_from\_env`, server branch) on origin/master.